Übungsblatt 9: C, Assembler und Code-Schloss

Mikrocomputertechnik 1

Musterlösung – Papieraufgaben und Sollverhalten für die Simulatoraufgabe.

Ziel und Voraussetzungen

Sie verbinden die Inhalte der Einheiten 1–8: C-Kontrollfluss, RV32I-Anweisungen, Aufrufkonvention, Datenpfad, Pipeline und Ein-/Ausgabe. Bearbeiten Sie die Papieraufgaben ohne Simulator und begründen Sie Ihre Ergebnisse. Die praktische Aufgabe prüft danach die gleiche Spezifikation mit Ripes.

Gemeinsame Annahmen: RV32I ohne komprimierte Befehle, 32-Bit-Register, vier Byte pro `int` im vereinbarten ILP32-Datenmodell. Vorzeichenbehaftete C-Zwischenresultate sind darstellbar. Normale Funktionsaufrufe verwenden `a0` für das skalare Ergebnis und einen 16-Byte-ausgerichteten Stack.

Papieraufgabe	Thema	Übungspunkte
1	Werkzeugkette und Zielauswahl	10
2	Kontrollfluss und Schleifentrace	12
3	Arrayadresse und vollständige Funktion	12
4	Aufruf und Stackframe	8
5	Zustand und Eingabeereignis	10
6	Datenpfad und Load-Use	8
Summe		**60**

Die Punkte dienen der Selbstkontrolle, nicht der Festlegung einer späteren Klausur. Für eine gemeinsame Einheit bieten sich die Aufgaben 1–3 und 5 sowie der praktische Basistest an. Aufgaben 4 und 6 und die Erweiterung vertiefen die Wiederholung.

Lehrdateien im Repository unter `beispiele/`:

- `vl9_control.c`: C-Beispiele
- `vl9_sum_array.s` und `vl9_calls.s`: manuelle Funktionen
- `vl9_next_state.s` und `vl9_code_lock.s`: Zustandsfunktion und Ripes-Programm

Die Funktionsdateien sind keine eigenständigen Programme. Das Schlossprogramm enthält dagegen Einstieg und alle Hilfsfunktionen. Compilerexperimente sind optional; ein fehlender Crosscompiler verhindert weder Papieraufgaben noch Ripes-Aufgabe.

Aufgabe 1: Werkzeugkette und Zielauswahl (10 Punkte)

Arbeitsauftrag

1. Ordnen Sie die vier Werkzeuge **Compiler, Linker, Präprozessor, Assembler** und nennen Sie zu jedem das erzeugte Zwischenergebnis. (4 P)
2. Welche Stufe ist jeweils die letzte bei `gcc -E`, `gcc -S` und `gcc -c`? (3 P)
3. Eine Assemblerdatei wurde mit `-march=rv32i -mabi=ilp32 -S` erzeugt. Erläutern Sie die beiden Zieloptionen und warum diese Datei allein noch kein startfähiges Programm ist. (3 P)

Verwenden Sie die Werkzeugnamen präzise: Assemblertext und das Werkzeug Assembler sind verschiedene Dinge.

Musterlösung

Werkzeug	Zwischenergebnis
Präprozessor	Präprozessierter C-Text, etwa `.i`
Compiler	Assemblertext, etwa `.s`
Assembler	Objektdatei, etwa `.o`
Linker	Verknüpftes Programm, etwa ELF-Datei

`-E` stoppt nach dem Präprozessor, `-S` nach dem Compiler mit Assemblertext, `-c` nach dem Assembler mit einer Objektdatei.

`-march=rv32i` wählt den 32-Bit-RISC-V-Basisbefehlssatz. `-mabi=ilp32` wählt die zugehörige Binärschnittstelle mit 32 Bit für `int`, `long` und Zeiger. Die Optionen setzen eine passende Toolchain voraus. Assemblertext muss erst assembliert und verknüpft werden. Außerdem müssen Programmeinstieg und Laufzeitumgebung passen; ein Header liefert beispielsweise nicht den Maschinenkörper einer Bibliotheksfunktion.

Aufgabe 2: Kontrollfluss und Schleifentrace (12 Punkte)

Arbeitsauftrag

a) Fehler finden (4 P). Das Fragment soll bei Gleichheit eins, sonst zwei in `a0` liefern.

```
bne a0, a1, Else
li  a0, 1
Else:
li  a0, 2
```

Verfolgen Sie die Fälle (5, 5) und (5, 4). Nennen und korrigieren Sie die fehlende Kontrollkante.

b) Schleife übersetzen (6 P). Schreiben Sie eine vollständige Blattfunktion `count_to_ten` einschließlich Rückkehr. `i` kommt in `a0` an, das Ergebnis bleibt in `a0`. Temporäre Register dürfen Sie verwenden.

```
int count_to_ten(int i) {
    while (i < 10) { i++; }
    return i;
}
```

Wie oft werden Bedingung und Körper bei Startwert 0 ausgeführt? Welches Ergebnis entsteht bei Startwert 12?

c) Vorzeichen (2 P). `a0` enthält `0xFFFFFFFF`. Welche Werte repräsentiert dieses Muster als vorzeichenbehaftete beziehungsweise vorzeichenlose 32-Bit-Zahl? Warum ist `bgeu` hier kein Ersatz für `bge`?

Musterlösung

a) (5, 5): erst eins, danach durch Fall-through zwei, also falsch. (5, 4): direkter Sprung zu Else, Ergebnis zwei. Nach `li a0, 1` fehlt `j End`, wobei `End:` nach dem Else-Zweig steht.

b) Eine gültige manuelle Lösung:

```
count_to_ten:
    li t0, 10
count_loop:
    bge a0, t0, count_end
    addi a0, a0, 1
    j count_loop
count_end:
    ret
```

Start 0: elf Bedingungsprüfungen, zehn Körperausführungen, Ergebnis 10. Start 12: eine Prüfung, kein Körper, Ergebnis 12.

c) Das Muster ist vorzeichenbehaftet −1, vorzeichenlos 4294967295. `bge` vergleicht vorzeichenbehaftet, `bgeu` vorzeichenlos. Für −1 soll die Schleife laufen; ein vorzeichenloser Vergleich würde sofort abbrechen.

Aufgabe 3: Arrayadresse und Summe (12 Punkte)

Arbeitsauftrag

Ein `int`-Array enthält `{10, 20, 30, 40, 50}` und beginnt im Adressmodell bei `0x2000`.

1. Berechnen Sie die Adresse und den Wert von `p[3]`. Liegt `0x2014` auf einem vorhandenen Element? Begründen Sie mit $A_i = B + 4i$. (4 P)
2. Ergänzen Sie die Funktion. `a0` enthält anfangs die Basisadresse, `a1` die vorzeichenlose Länge. `t0` hält die Basis, `t1` den Index, `t2` die Summe. Tragen Sie jeweils den **fehlenden Befehlsnamen** in die fünf Lücken ein. (5 P)
3. Nennen Sie Ergebnis und Anzahl der Array-Lesezugriffe für `n=0` und `n=5`. Welche zusätzliche Voraussetzung braucht der Vergleich mit einer vorzeichenbehafteten C-Summe? (3 P)

```
sum_array:
    mv t0, a0
    li t1, 0
    li t2, 0
sum_array_loop:
    _______ t1, a1, sum_array_end
    _______ t3, t1, 2
    _______ t4, t0, t3
    _______ t5, 0(t4)
    add t2, t2, t5
    addi t1, t1, 1
    j sum_array_loop
sum_array_end:
    _______ a0, t2
    ret
```

Musterlösung

1. $A_3 = \texttt{0x2000} + 12 = \texttt{0x200C}$, Wert 40. `0x2014` wäre die Adresse des Index 5 und liegt direkt hinter dem Array. Sie darf hier nicht als vorhandenes Element gelesen werden.
2. In Reihenfolge: `bgeu`, `slli`, `add`, `lw`, `mv`. Der vorzeichenlose Vergleich passt zur Länge und zum Index. Die Verschiebung um zwei bildet den Byteoffset $4i$. Die komplette Lehrlösung steht in `beispiele/vl9_sum_array.s`.
3. `n=0`: Ergebnis 0, kein Array-Lesezugriff. `n=5`: Ergebnis 150, fünf Array-Lesezugriffe. Alle gelesenen Elemente müssen gültig und jede vorzeichenbehaftete C-Zwischensumme darstellbar sein. Maschinenaddition modulo 2^{32} definiert keinen C-Überlauf.

Aufgabe 4: Aufruf und Stackframe (8 Punkte)

Arbeitsauftrag

`calc` ist die Blattfunktion `slli a0, a0, 1` gefolgt von `ret`. Sie verändert keine gesicherten Register. Die folgende Funktion wird normal aufgerufen. Der eingehende `sp` ist `0x2000`.

```
double_plus_one:
    addi sp, sp, -16
    sw ra, 12(sp)
    call calc
    addi a0, a0, 1
    lw ra, 12(sp)
    addi sp, sp, 16
    ret
```

1. Berechnen Sie `sp` nach dem Prolog, die Speicheradresse von `ra` und `sp` nach der Wiederherstellung. (3 P)
2. Warum wird `ra` gesichert? Weshalb sind 16 Byte reserviert, obwohl nur ein Wort gesichert wird? (3 P)
3. Ein Aufrufer benötigt seinen Zähler in `t0` nach einem normalen Funktionsaufruf erneut. Wer ist gemäß ABI für die Sicherung verantwortlich? Berechnen Sie außerdem das Ergebnis für `a0=-3`. (2 P)

Musterlösung

1. Neuer `sp`: `0x1FF0`. Sicherungsadresse: `0x1FF0 + 12 = 0x1FFC`. Nach dem Epilog: `0x2000`.
2. `call calc` überschreibt `ra`, die Rücksprungadresse. Die Funktion braucht ihre eingehende Adresse später für `ret` zum ursprünglichen Aufrufer. 16 Byte erhalten die vereinbarte Stackausrichtung. Der Nutzwert belegt nur vier Byte.
3. `t0` ist caller-saved: Der Aufrufer muss einen noch benötigten Wert selbst sichern. `calc(-3)=-6`, anschließend Addition eins: Ergebnis –5. Die Aussage über Registererhaltung gilt für normale ABI-Aufrufe, nicht pauschal für beliebige `ecall`-Umgebungen.

Aufgabe 5: Zustandsmaschine und Ereignis (10 Punkte)

Gemeinsame Schlossregeln

S0 ist rot, S1 gelb, S2 blau, S3 grün. Die Folge UP, DOWN, RIGHT öffnet das Schloss. Falsche einzelne Codetasten setzen S0–S2 nach S0 zurück. S3 bleibt bei Codetasten offen. LEFT setzt jeden Zustand nach S0 zurück. Ereignis 0 erhält gültige Zustände, ein ungültiger Zustand wird nach S0 korrigiert.

Rohmasken: UP=1, DOWN=2, LEFT=4, RIGHT=8. Eine leere Gruppe gibt die Eingabe frei. Die nächste nichtleere Gruppe verbraucht diese Freigabe. Nur eine einzelne Taste erzeugt ein Ereignis: UP=1, DOWN=2, RIGHT=3, RESET=4. Mehrfachdruck erzeugt Ereignis 0 und erfordert ebenfalls vollständiges Loslassen.

Arbeitsauftrag

1. Ergänzen Sie die Übergänge. (4 P)
2. Vervollständigen Sie den Trace. Anfang: S0, Freigabe geschlossen. Jeder Wert bezeichnet eine Abfragegruppe; der Zustand steht **nach** ihrer Verarbeitung. (4 P)
3. Was muss bei Zustandswert 99 und Ereignis 0 geschehen? Warum reicht ein Sprung zu einer Marke namens `state0` nicht? (2 P)

Musterlösung

Aktuell	Kein Ereignis	UP	DOWN	RIGHT	RESET
S0	S0	S1	S0	S0	S0
S1	S1	S0	S2	S0	S0
S2	S2	S0	S0	S3	S0
S3	S3	S3	S3	S3	S0

Rohmaske	0	1	1	0	3	2	0	2	0	8	0	4
Ereignis	0	1	0	0	0	0	0	2	0	3	0	4
Zustand	S0	S1	S1	S1	S1	S1	S1	S2	S2	S3	S3	S0

Die Mehrfachmaske 3 verbraucht die Freigabe. Der danach noch gehaltene DOWN-Wert 2 zählt nicht. Erst nach null kann ein neuer DOWN-Druck S2 erreichen.

Die Übergangsfunktion muss null liefern, und der Aufrufer muss null im Zustandsregister speichern. Eine Sprungmarke verändert allein den Programmzähler, nicht den dort gespeicherten Zustandswert.

Aufgabe 6: Datenpfad und Load-Use (8 Punkte)

Arbeitsauftrag

a) Hauptsteuerung (3 P). Im Einzyklus-Teilmodell aus Einheit 5/6: Nennen Sie für `sw` die Werte von `RegWrite`, `ALUSrc` und `MemWrite`. Begründen Sie, warum `MemtoReg` dort als `X` markiert werden darf.

b) Pipeline (5 P). Zeichnen Sie die Stufen für das Fragment ein:

```
lw  t0, 0(a0)
add t1, t0, t0
```

Verwenden Sie das Kursmodell: IF/ID/EX/MEM/WB, Forwarding von EX/MEM und MEM/WB, ein Load-Use-Stall. Die Registerbank darf dasselbe Register nicht im gleichen Takt schreiben und lesen. Der gelesene Speicherwert sei 42.

Kennzeichnen Sie den Stall, die Bubble und den Weiterleitungspfad. Nennen Sie Gesamttaktzahl und Endwert von `t1`.

Musterlösung

a) `RegWrite=0`, `ALUSrc=1`, `MemWrite=1`. `sw` berechnet Basis plus Immediate und schreibt den Registerwert in den Speicher. Da kein Zielregister beschrieben wird, hat die Wahl des Register-Rückschreibwerts `MemtoReg` keinen Einfluss. Ein `X` erlaubt keinen unbestimmten Schreibzugriff.

b)

Anweisung / Takt	1	2	3	4	5	6	7
`lw`	IF	ID	EX	MEM	WB		
`add`		IF	ID	ID gehalten	EX	MEM	WB
Bubble in EX				Bubble			

IF: Instruction Fetch, ID: Instruction Decode, EX: Execute, MEM: Memory Access, WB: Write Back. Der Load-Wert liegt erst nach MEM in Takt 4 vor. Deshalb wird `add` einen Takt gehalten und der Wert 42 in Takt 5 von MEM/WB zu **beiden** ALU-Eingängen weitergeleitet. `t1=84`, Gesamtdauer sieben Takte. Die Bubble ist keine vom Programm eingefügte `nop`-Anweisung.

Aufgabe 7: Code-Schloss mit Ripes

Vorbereitung und Geräteprüfung

1. Wählen Sie ein geeignetes RV32I-Prozessormodell und richten Sie **ein D-Pad und eine LED-Matrix** ein. Notieren Sie Ripes-Version und Prozessormodell.
2. Prüfen Sie die exportierten Symbole `D_PAD_0_BASE` und `LED_MATRIX_0_BASE`. Die tatsächlichen Basisadressen werden vom Gerätelayout bestimmt. Bei abweichenden Namen passen Sie nur diese Symbolverwendungen in Ihrer Arbeitskopie an.
3. Prüfen Sie die D-Pad-Wortoffsets UP=0, DOWN=4, LEFT=8, RIGHT=12 und das Farbformat des ersten LED-Pixels. Der dokumentierte Kursstand verwendet `0x00RRGGBB`.
4. Öffnen Sie `beispiele/vl9_code_lock.s`. Das Programm enthält alle Funktionen; zusätzliche `include`-Dateien sind nicht erforderlich.

Geräteannahme: Im dokumentierten Ripes-Quellstand liefert ein D-Pad-Lesen eine gehaltene oder seit dem letzten Lesen erfasste Betätigung und löscht den gespeicherten Druckindikator. Das Programm liest deshalb jedes Richtungsregister einmal pro Gruppe. Vier aufeinanderfolgende Lesezugriffe sind keine atomare Momentaufnahme. Prüfen Sie dieses Verhalten an Ihrer Installation. Die Freigaberegel ersetzt keine physikalische Entprellung.

Durchführung

1. Starten Sie ohne gedrückte Taste. Beobachten Sie S0 und rot. Drücken und lösen Sie UP, DOWN und RIGHT nacheinander. Zwischen Betätigungen muss mindestens eine vollständig leere Abfragegruppe verarbeitet werden.
2. Halten Sie UP in einem neuen Versuch. Prüfen Sie, dass der Zustand nicht durch mehrere UP-Ereignisse weiter verändert wird.
3. Erzeugen Sie nach einer leeren Gruppe eine Mehrfachmaske und lassen Sie nur eine Taste los. Erst vollständiges Loslassen darf die nächste neue Betätigung freigeben. Wenn die Oberfläche einen Mehrfachdruck nicht erlaubt, verfolgen Sie diesen Fall im Debugger mit passenden Rohmasken.
4. Prüfen Sie S1 mit neuem UP, S3 mit neuem DOWN und LEFT als neuen Druck in jedem Zustand.
5. Halten Sie das Programm **vor `mv a0, s0` bei `update`** an. Setzen Sie `s0=99` und `a1=0`. Führen Sie die Übergangsfunktion und Rückkopie schrittweise aus. Prüfen Sie `s0=0` und die rote Ausgabe.

Erfassen Sie pro Test **Rohmaske, Freigabe, Ereignis, Zustand und Farbe**. Vergleichen Sie zuerst mit Ihrer Papierlösung, dann mit der Musterlösung. Ein sichtbarer Farbwechsel allein belegt nicht die Registerkorrektur im Zusatzfall.

Aufgabe 7: Protokoll und Erweiterung

Testprotokoll

Test	Soll	Beobachtung / erste Abweichung
Start, keine Taste	S0, rot	
UP → DOWN → RIGHT	S1 → S2 → S3	
UP halten	Ein neues UP-Ereignis	
Mehrfachdruck, teilweise lösen	Kein neues Ereignis	
Neuer UP in S1	S0	
Neuer DOWN in S3	S3	
Neuer LEFT	S0	
`s0=99`, Ereignis 0	`s0=0`, rot	

Musterlösung und Funktionsaufteilung

Das Sollverhalten steht in der Tabelle. Ein Simulatorprotokoll muss aus dem eigenen Lauf stammen; diese Musterlösung enthält keine erfundenen Beobachtungen.

`read_sample` liest die vier Richtungen und bildet die Rohmaske. Die Hauptschleife hält die Freigabe in `s3`, beginnt gesperrt und akzeptiert nur Einzelmasken nach einer leeren Gruppe. Auch eine Mehrfachmaske schließt die Freigabe. `next_state` liefert den Folgezustand in `a0`. `mv s0, a0` speichert ihn. `state_colour` liefert das RGB-Wort, das `sw a0, 0(s2)` ins erste Pixel schreibt.

Alle Hilfsfunktionen sind Blattfunktionen und erhalten `s0–s11` sowie `sp`. Die Hauptschleife läuft dauerhaft und verwendet keinen Exit-Dienst.

Erweiterung: Andere Codefolge

Ändern Sie in einer Arbeitskopie die Folge zu **DOWN, UP, RIGHT**. Lassen Sie Eingabefreigabe, Rohmasken und LEFT-Rücksetzen unverändert. Schreiben Sie zuerst die geänderte Übergangstabelle und ändern Sie danach die Zustandsfunktion. Die Originaldateien dienen weiter als Referenz.

Lösungshinweis: S0 führt mit DOWN nach S1, S1 mit UP nach S2, S2 mit RIGHT nach S3. Die sonstigen falschen Codetasten in S0–S2 führen weiterhin nach S0. In den Marken `from_s0` und `from_s1` werden die erwarteten Ereignisse von 1 nach 2 beziehungsweise von 2 nach 1 geändert. Die Eingabeerkennung wird nicht umcodiert. Prüfen Sie neue Erfolgsfolge, alte Folge, Halten, Mehrfachdruck und Rücksetzen.

Optional: Reale Compilerübersetzung vergleichen

Vergleichen Sie eine tatsächliche RV32I-Übersetzung von `vl9_control.c` mit `-O0` und `-O2` in einer vorhandenen passenden Toolchain. Verwenden Sie `-S -march=rv32i -mabi=ilp32` und dokumentieren Sie Version und vollständigen Aufruf. Alternativ konfigurieren Sie das entsprechende Ziel in Compiler Explorer.

Erklären Sie eine Optimierung am tatsächlich erzeugten Text. Ohne Compilerlauf dürfen Sie keine Befehlsfolge als dessen Ausgabe darstellen. Assemblertext allein belegt weder Linkbarkeit noch einen Ripes-Lauf.

Auswertung: Der konkrete Text hängt von Toolchain und Optionen ab. Bei `constant_sum` ist beispielsweise eine Zusammenfassung der Rechnung zur Konstanten 15 möglich. Ein Vergleich muss trotzdem die tatsächlich erzeugten Dateien heranziehen. Eine allgemeine Zusicherung bestimmter Register oder Befehlsfolgen ist keine Musterlösung für diesen Versuch.